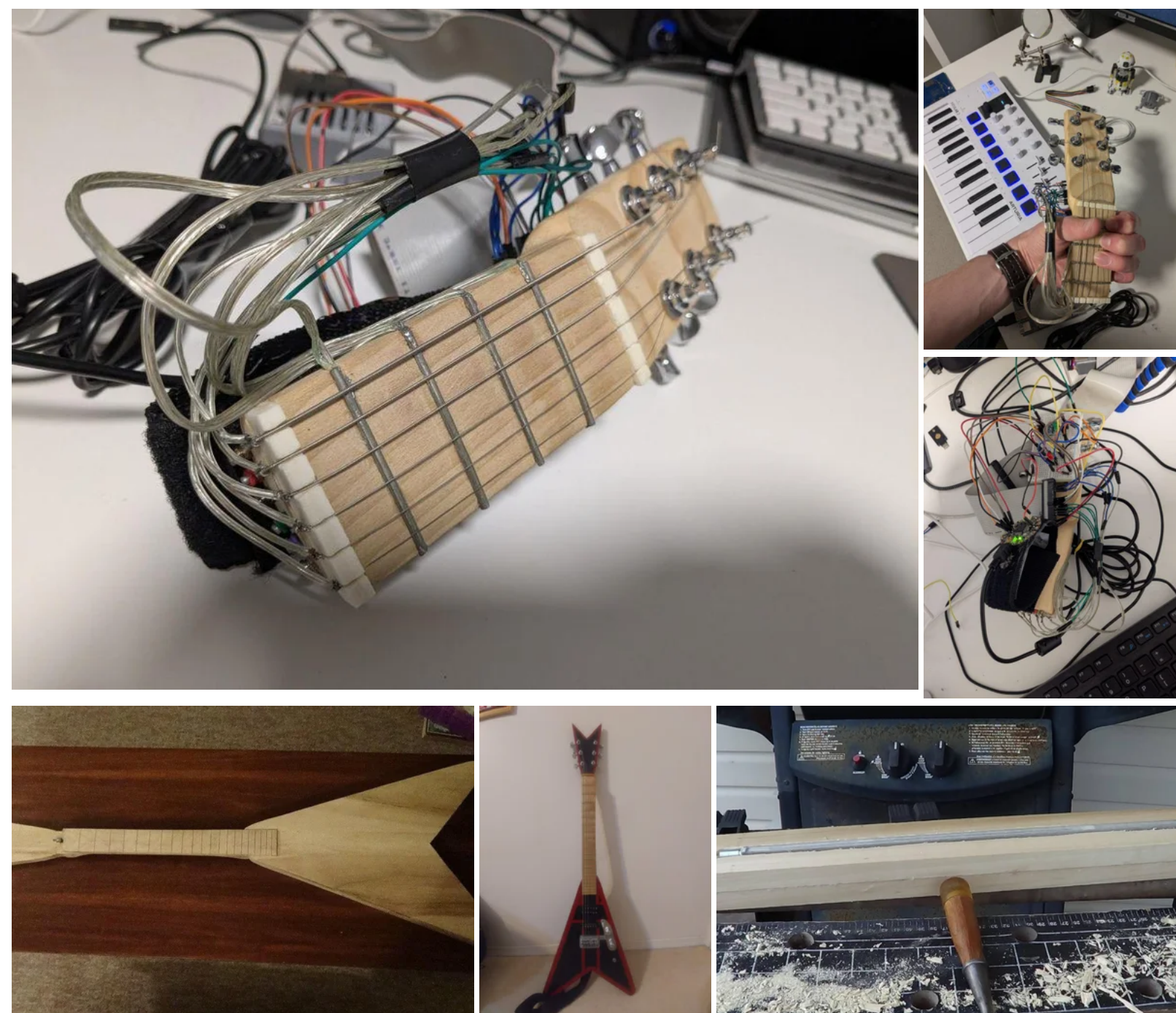


Raspberry Pi and Flora Based Guitar MIDI Controller

By [marcusstephenedwards](#) in [LivingMusic](#)
Published Jan 17th, 2026



Introduction: Raspberry Pi and Flora Based Guitar MIDI Controller





Welcome to your next musical instrument building adventure!

The Hobby

My instrument building hobby started mostly working with hand tools on the deck in the summertime. I was in a woodworking club at the time. I often referred to "Guitarmaking: Tradition and Technology" by William Cumpiano and Jonathan D. Natelson. A great book.

I included a few photos of my favourite flying V here. This was the first one I built! It was not the last flying V I made. It was made of maple (hardwood) and took a lot of time and effort to work with. I did use shop tools for this one, like belt sander, planer and band saw. None of these shop tools will be required for today's project!

Next, I built a fretless guitar neck. I did this project at home and with hand tools. I used chisels inherited from my Opa, who was a carpenter. I got a Japanese saw rasp from Lee Valley and carved out the neck. I even made an [instructable](#) about it.

After these, I built a black flying V (for my dad) and a white bass guitar (for my now-wife... it was a good idea, I guess).

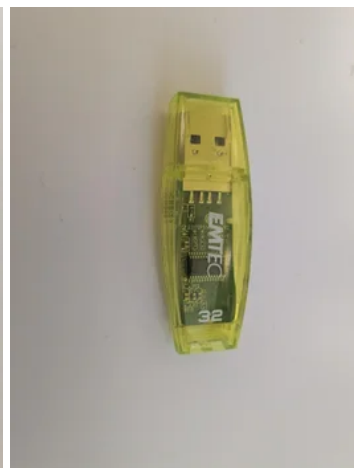
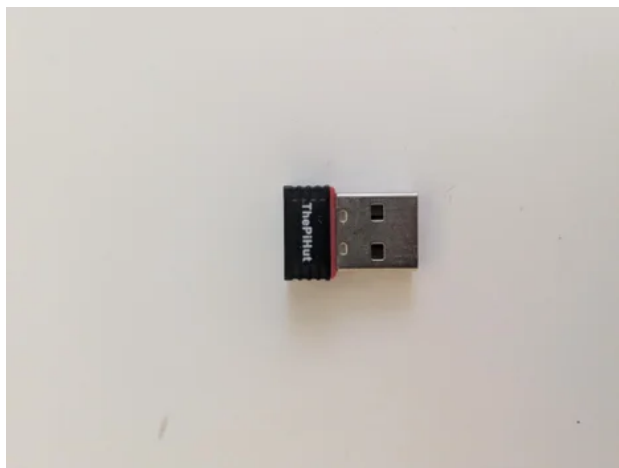
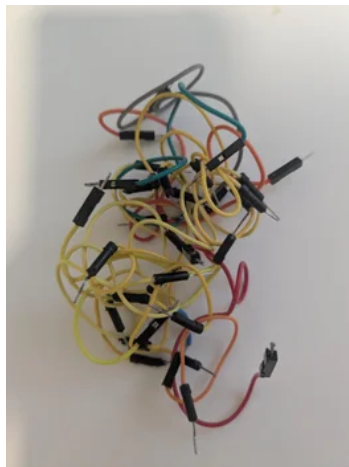
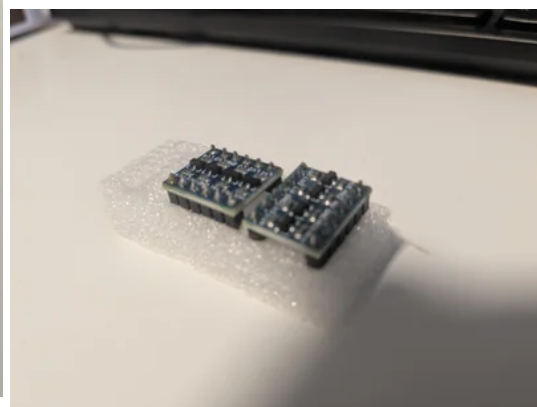
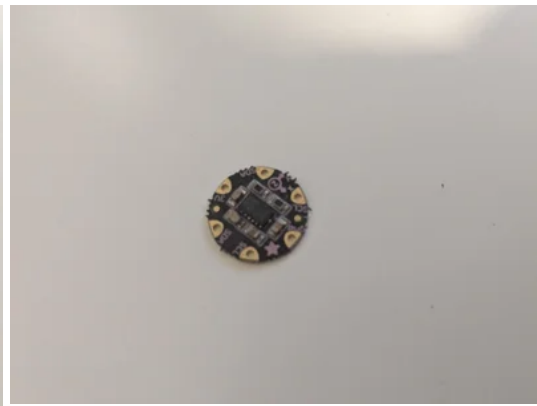
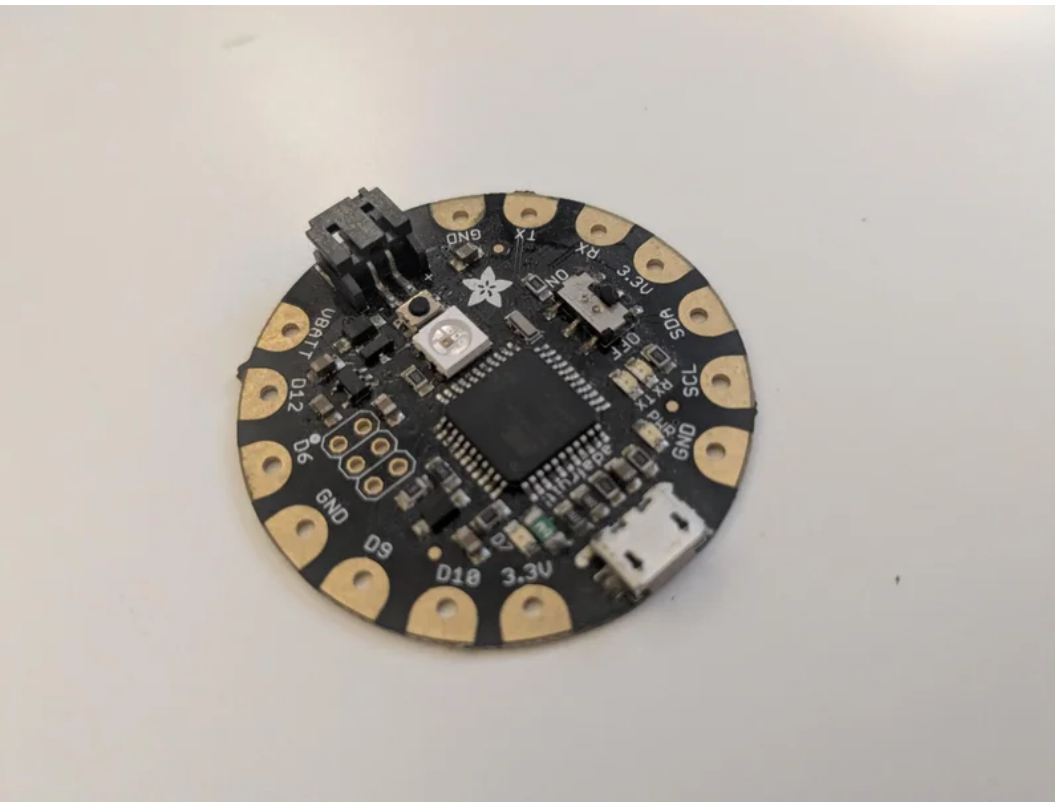
This Instructable

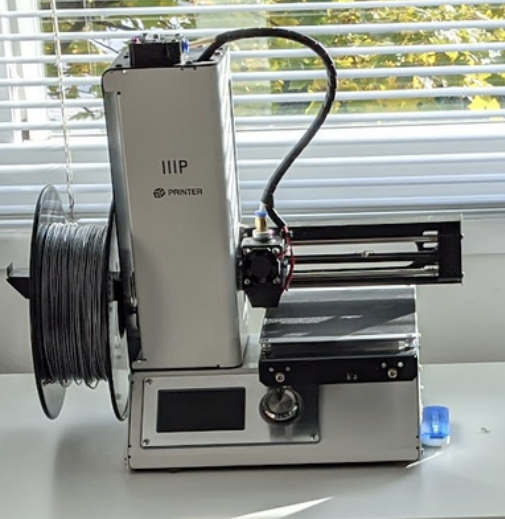
The Guitar MIDI Controller you'll learn to make in this instructable is a hand-made, custom MIDI controller made from a piece of pine wood, a Raspberry Pi 2 model B, wearable Adafruit Flora and a LSM303 accelerometer module. After building a few traditional guitars and having a lot of fun with the fretless one, I felt inspired to do something new.

Wherever you are in the hobby, this should be a good project for you. If you are an experienced guitar builder or Luthier then the woodworking will be boring to you, but you might learn a bit about electronics. Conversely, if you are an experienced maker but this is your first musical instrument, it is a great soft introduction to guitar building. Just to get your feet a little bit wet.

Maybe you're already an expert in everything. That is fine too! Then this shouldn't be challenging for you, but should still be tons of fun!

Supplies







This project doubles as a woodworking and electronics project (with a little software, 3D printing and sewing mixed in)! Therefore, the supplies are broken into a few categories.

Electronics

1. AdaFruit Flora
2. Raspberry Pi
3. Wearable LSM303 accelerometer module
4. 4-bit digital logic level converter
5. Prototyping wire
6. Half-length breadboard
7. 40-pin SATA cable
8. Conductive thread

Guitar Parts

1. Guitar strings
2. Guitar tuning heads
3. Two guitar nuts
4. Guitar fret wire
5. Wood of choice

Computer Components

1. Computer
2. USB charger x 2
3. USB to SD card adapter
4. USB drive
5. 2 SD cards
6. Mouse
7. Keyboard
8. Monitor
9. HDMI cable
10. PiHut USB WiFi adapter

11. Aux cord
12. Speakers or amplifier

Tools

1. Wire cutter
2. Wire stripper
3. Electrical tape
4. Soldering iron
5. 3D printer
6. Pencil
7. Protractor
8. Ruler
9. Sewing needle
10. Radius sanding block
11. Coarse grain sandpaper
12. Fine grain sand paper
13. Drill with bit
14. Wood glue
15. Hand saw
16. Wood clamps
17. Rasp
18. Saw rasp

Software

1. Arduino IDE
2. Adafruit LSM303 accel library
3. Cura
4. Trixie OS
5. Fluidsynth
6. pulseaudio

Miscellaneous

1. Rigid and sturdy fabric armband
2. 6 small nails

Step 1: Woodworking Materials Preparation

The first component of this project is the woodworking.

Setting Expectations

This part will be much easier than building a full guitar neck. There is hardly any length to the MIDI guitar's neck so there is no need to worry about bend or a truss rod. The spacing of the frets also doesn't really matter, as the resonances of the strings are not used by this instrument to make the individual notes. You don't need to worry about neck taper, either.

In terms of materials, you don't need to worry about using any expensive hardwood. I built mine from pine, which is a softwood. There is so little material needed that you can probably get away with using shop scraps.

Even tuning up the guitar is easier. There is no need to keep this one in tune, the only requirement is that the strings are taught enough that they don't bump into each other, and loose enough that they aren't touching the frets all the time.

Measuring and Preparing Materials

For this project, you will need a piece of wood that is about 2" x 3" x 8". For a typical guitar neck, you need to be certain that the neck is carved from one piece of hardwood. This is so that the neck does not have any weak points where it will break under the tension of the strings. However, for this project, there will be virtually no tension from the strings or non-existent truss rod. so, if you need to, you can get away with gluing a few pieces of wood together.

For the fretboard, we will still use a separate piece. You could use a material that looks nice for the fingerboard, like rosewood. The material should be around 2" wide, 5", and a few millimeters thick.

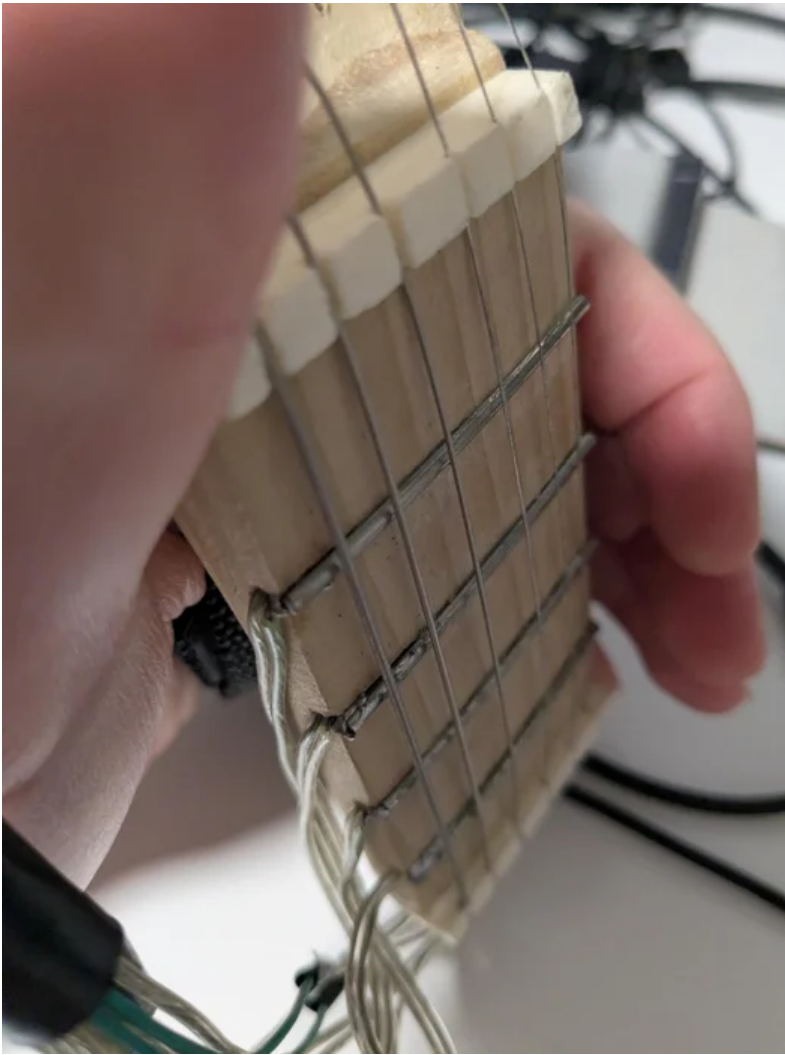
Draw four lines across the fretboard material, orthogonal to the edges. Space them roughly equally. The main thing is to make sure that it is comfortable to place each of your four fingers on your playing hand in its own division. Your fingers should be to the left (right) of the lines if you are playing with your left (right) hand.

Leaving a little space after the last fret position, and enough space for your first finger before the first, you can draw lines where you will cut the ends of the mini fretboard.

Next, we will look at the neck / head piece. The neck will be just as wide and a little longer than the fretboard. Take both of your guitar nuts and place them against the end lines of the fretboard on either side. The length of the fretboard plus both nuts will be the total length of the neck. Mark this on one side of the neck / head blank.

On the front of the neck / head blank, we will mark where the head will start. The head will be half an inch thick, around three inches long and come out at a 15 degree angle from the neck. Start a centimeter away from the edge of the fingerboard, and draw a line across the blank. This is where we are going to start the head. On the sides of the blank, measure a 15 degree angle below the top surface and draw a line at this angle with your pencil, starting from the line you just drew on the front. Draw another parallel to the first, half an inch below it. Extend this line from the end of the head to just within one inch of the front of the neck.

Step 2: Making the Fretboard



Cut along the line at the end of your fretboard with your saw. Then sand the end so that it is smooth. This is your fretboard. Next, we will want to add the frets. This is going to work a bit differently from how the frets are added to a typical guitar neck.

Cut shallowly along the lines where you drew your fret positions. You will notice your fret wire has an insert that is meant to sit within these cuts, beneath the surface of the fretboard. Your cuts will have to be slightly deeper than this, to allow us to also lay down some stripped wire beneath the frets.

Once you have made these careful cuts with your saw, strip some wire so that it will be able to make good contact with the fret itself. We will solder the wire to the fret on the ends, so no need to worry about contact being perfect at this stage. The ends are what we care about.

Make sure each wire is long enough that it can extend at least a few inches beyond the side of the guitar neck, and is only stripped long enough that we can solder both ends of the fret to it.

Next, get your fret wire itself and cut it with wire cutter into four pieces that are the width of the fretboard. Use a rasp to smooth the sharp edges so that you don't hurt yourself when playing the instrument later.

Put wood glue in a fret cavity. Then place the stripped end of one of the electrical wires in the cavity with the glue, making sure that there is something we can solder to on either end. Finally, place a fret on top of it all and clamp it into place while it dries. Repeat this three more times for the other frets.

Once we are done gluing the frets, take out your soldering iron and solder the ends of the wires to the side of the frets. The glue should hold the wires sturdily in place, and the solder is only for an electrical connection.

Before rasping out the shape of the neck, glue the fretboard onto the front of the neck / head blank, leaving room for a nut on either end of the neck. Glue the nuts on as well. Make sure the nuts are oriented the same way. Clamp and wait for the glue to fully dry.

Step 3: Cutting the Head and Carving the Neck



Cutting out the Head

Cut along each of the parallel lines you drew before to create your angled head blank. Cut the scrap wood that is created by this off fully, without cutting into the neck area. Once this is done, you can take your drill and drill six holes of the appropriate radius for your tuning heads into the head. They should be equally spaced along the head, each one slightly inset slightly further so that the strings do not touch one another once the instrument is strung.

Carving out the Neck

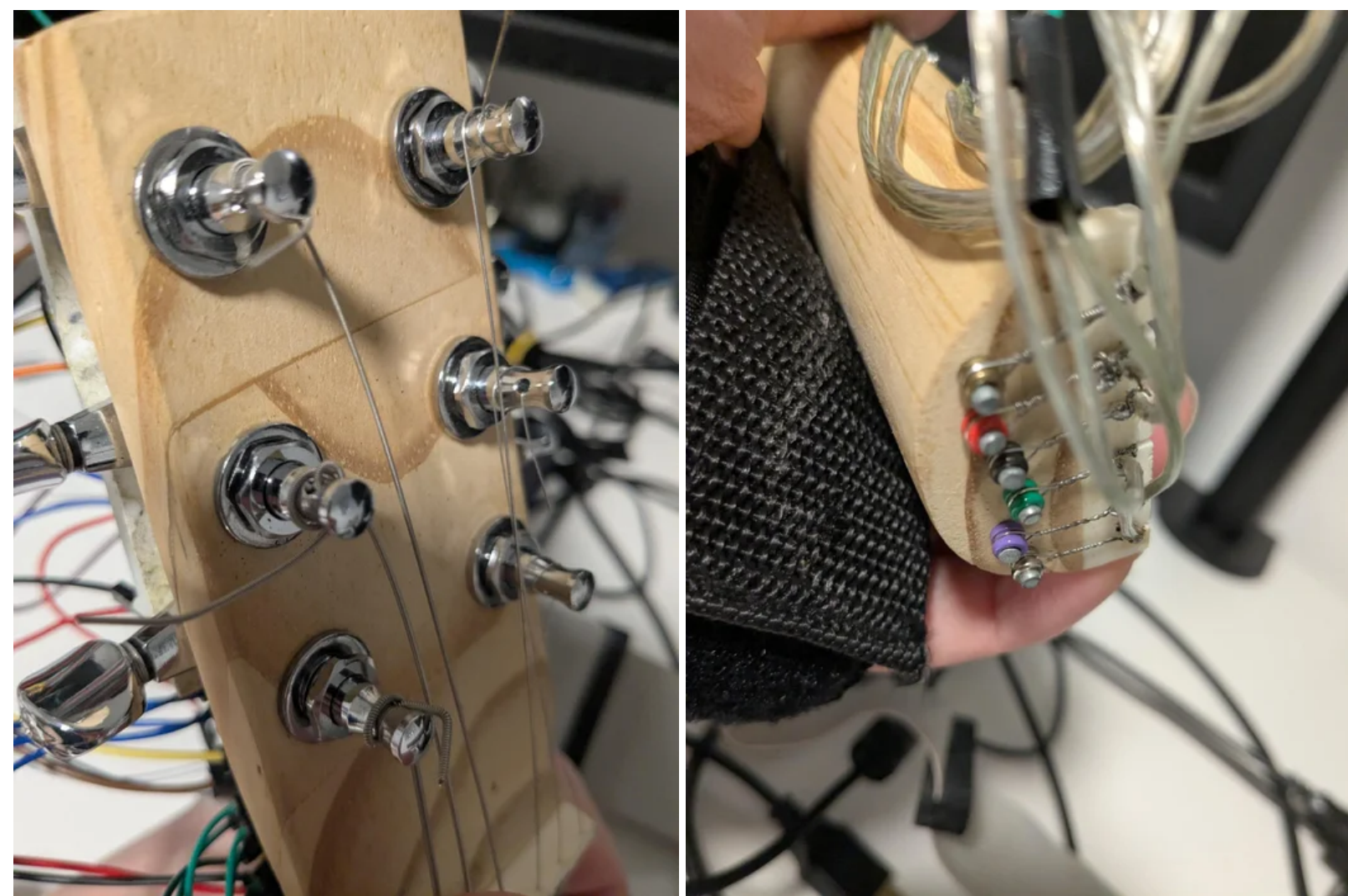
Carving out the neck is normally a time consuming part of building a guitar from scratch. It should not take long for you to carve out this stubby four-fret neck, however!

Rasp out a semicircular shape from the neck blank that feels comfortable in your playing hand, and allows you to reach all the positions on the fretboard. Try to smoothly transition between the neck and head, keeping in mind that it will still be sanded later.

Finishing Touches

Once the rough shape of the instrument is complete, sand everything down until it feels smooth in your hand and all the edges look nicely rounded.

Step 4: Stringing, Etc.



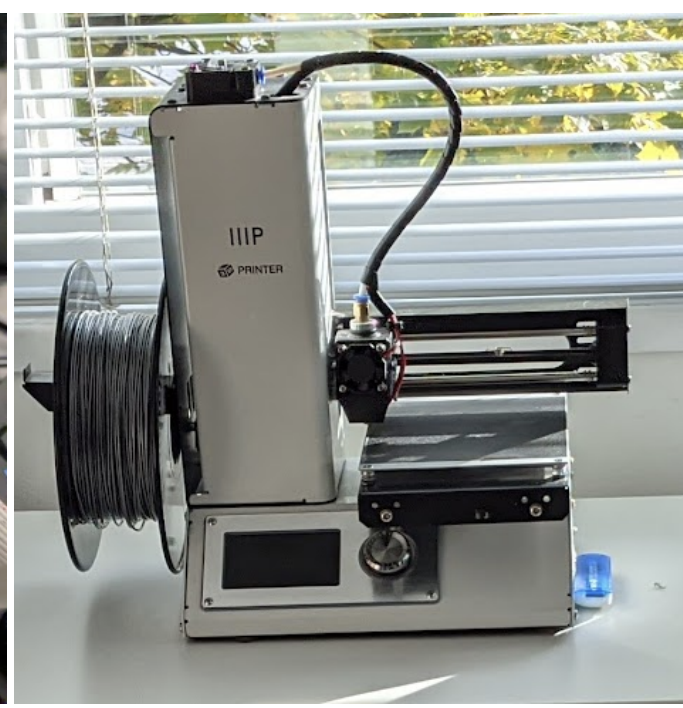
Stringing the Instrument

Next, you can install the tuning heads and string the instrument. String the instrument the same way as you would string a guitar, except that you will cut the strings very short. That, and the strings should not be tightened very much or tuned. Just tighten enough that they hover above the frets. Instead of a saddle holding the strings on one end, you can nail the strings into the end of the instrument permanently (they are unlikely to break or wear out).

Additional Parts

Take your rigid armband, SATA cable and half-length breadboard and attach them to the instrument in the following ways. The armband should be attached to the back of the neck. To do this I used two nails which I first drove through two small (few cm x few cm) squares of metal sheet, then the armband, then the neck. Carefully separate two adjacent wires on the SATA cable and use a nail and a washer to affix one end of the cable between the armband and the head of the MIDI guitar controller. Finally, you can hot glue the half-length breadboard to the backs of the tuning heads.

Step 5: 3D Printing a Case for the RPi



The final part of building the physical instrument is making a case for your Pi. We don't want it just sitting there on your workbench or pedalboard.

You can find lots of neat RPi cases on thingiverse.com. Just make sure that the one you choose allows access to the GPIO pins, since we need them for this project. Once you have chosen a case that you like, download the .stl file and open it up in CURA.

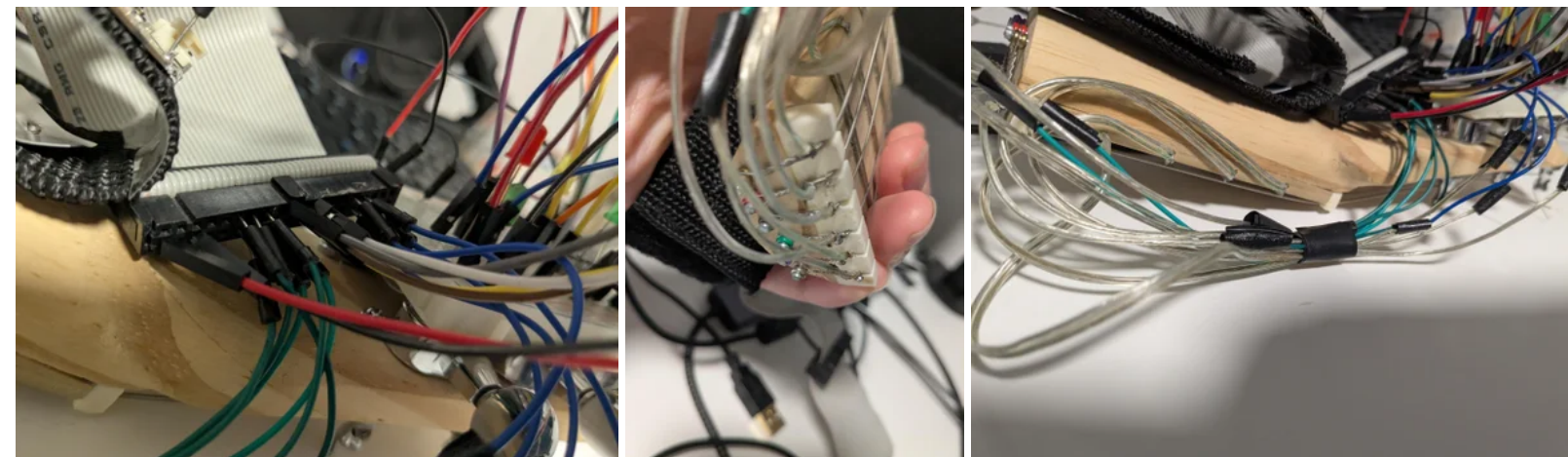
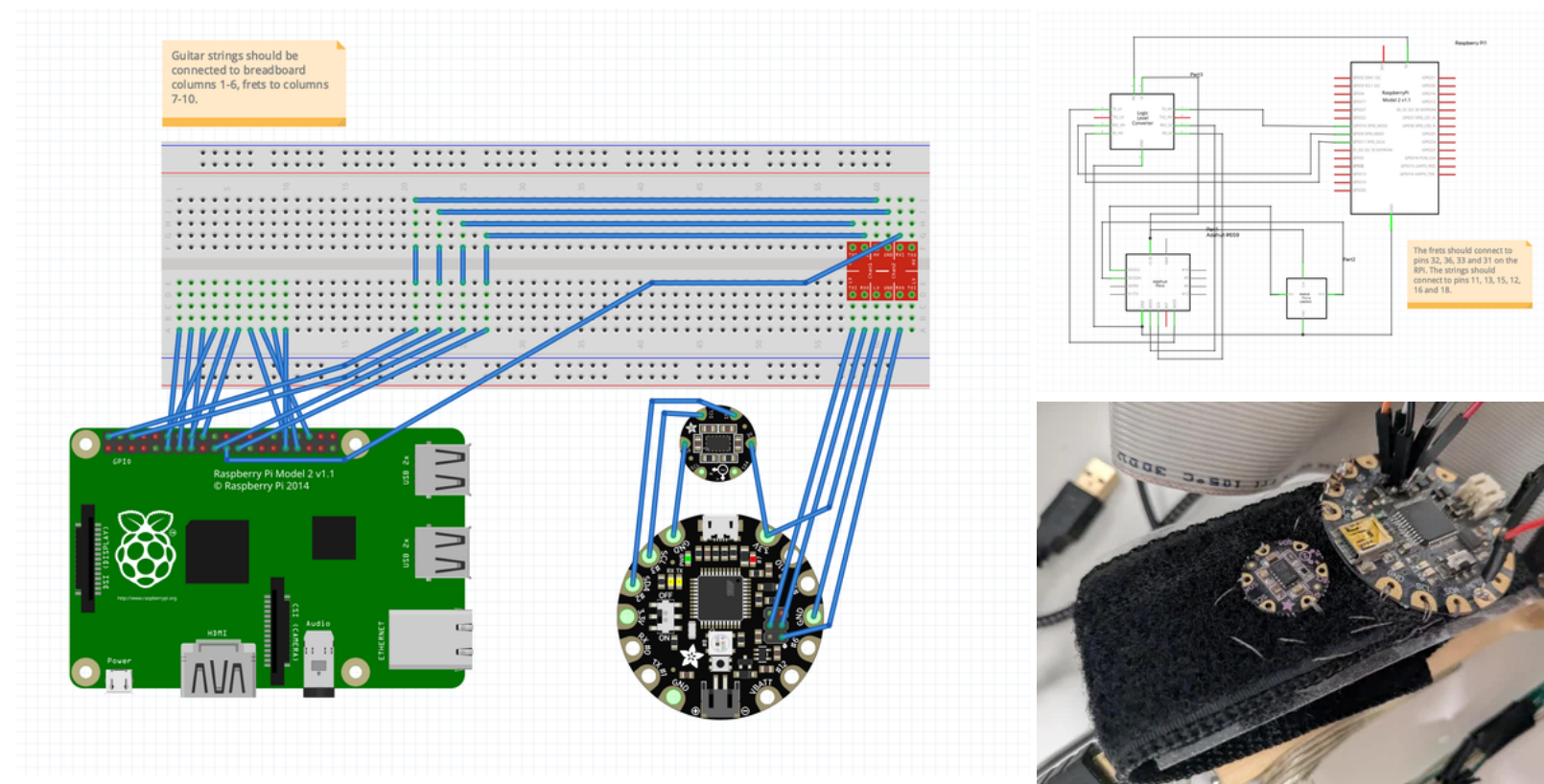
In CURA, you will need to select your filament type and print bed size. You will want to arrange the print on the bed. Next, you'll slice the model. This converts it to G-code which the 3D printer can understand. G-code is basically a list of motions for printer's stepper motors to make. You can preview the print in CURA before printing.

To print, I usually save my G-code to an SD card and then move that card over to the printer which has an interface for choosing prints in the mounted drive. Different printers work differently, you can also print directly from your computer if the printer is connected via USB.

You will want to monitor the print to make sure that it is going well, and restart if it has issues i.e. it doesn't stick to the bed, filament gets stuck in the extruder, etc. You might want to adjust the bed heat or the extruder heat until you get it working right. You may also need to unjam or replace the filament.

Finally, you'll remove the print from the bed and put your RPi snugly inside!

Step 6: Wiring



Now that we have built the physical part of the instrument, we need to instrument it. Next is wiring.

I've made wiring diagrams in Fritzing and included them with this section. There are a few different *types* of wires we will use, however. For connections between the Flora, logic level converter and RPi we will use prototyping wire. For the connections between the Flora and the LSM303, we will use conductive thread sewn into the rigid armband. For connections from the frets themselves, we will twist and solder together whatever wire you stripped and glued into the fretboard with prototyping wire so that one end can be inserted into the breadboard or directly into the SATA cable. We can do something similar for connections from the strings.

When working with the electronics, I recommend wearing a grounding strap so that no static discharge ruins your precious devices.

Now that you are wearing a grounding strap and have all your materials ready, we can start with the wiring.

Wiring the Flora and LSM303

You can begin by sewing the Flora into the armband by the 3.3V, GND, SCL and SDA terminals, using a sewing needle. Place the wearable LSM303 module where there is room for it, below the Flora, and sew towards it from the Flora's

terminals next. Make sure to connect the Flora's GND to the LSM303's gnd, the 3.3V to 3V, SCL to SCL and SDA to SDA. Wrap the thread around the terminals a few times before tying it off and cutting away any extra thread.

We aren't done with the Flora yet. We will make use of its ICSP header for the Serial Peripheral Interface (SPI) protocol in this project. You'll solder prototyping wire to each of three pins of the ICSP header. It can be a little crowded. I would use a different colour for MISO, MOSI and SCK. you don't need to use the RESET, +5V or GND pins on the ICSP header.

After that, we will need to solder a black prototyping wire to a remaining GND terminal and a red prototyping wire to an unused 3.3V terminal of the Flora. These two wires will be used to power the logic level converter on the low power side.

Wiring the Logic Level Converter

Connect the MISO, MOSI, SCK, 3.3V and GND wires from the Flora to the appropriate pins of the logic level converter on the breadboard.

On the other end of the converter, wire the high voltage pin to pin 2 or 4 of the RPi 's GPIO pins, using board numbering. These are both 5V pins. Use prototyping wire from the breadboard to the SATA connector for this. You don't need to connect the other end of the SATA cable to the RPi GPIO just yet, but imagine it is there. Wire the ground pin of the converter to any ground pin on the RPi. Pin 6 will do.

We also need to connect the MOSI (pin 19), MISO (pin 21) and SCLK (pin 23) pins of the RPi to the appropriate matching pins on the high-voltage side of the logic level converter. I recommend using the same colour wires for MOSI, MISO and SCLK on the Pi as you did for MOSI, MISO and SCK on the Flora. That way, you should have wires of like colour across from one another on the logic level converter.

We need a logic level converter because the RPi operates at 5V while the Flora operates on 3.3V. Hooking them up directly could fry the Flora!

Wiring the RaspberryPi to the Frets and Strings

Take six electrical wires and twist them around your strings, close to where they are nailed into the base of the instrument. Solder each wire to one string so that a good electrical connection is made. On the other ends of these six wires, strip them long enough so you can twist each of them around a prototyping wire that also has its end stripped. Solder each one together with its corresponding prototyping wire.

Similarly, take the four wires that are embedded in the fretboard and strip their ends. Twist and solder each of them to a prototyping wire. Now, you should have ten prototyping wires that are each connected on one end to a fret or string. I recommend using one colour of wire for frets and another colour for strings, to keep them visually distinguishable.

These ten wires can be plugged directly into the SATA, or into the breadboard first if having a second prototyping wire length gives you extra travel that you might benefit from. Having the breadboard in the path can also be helpful for debugging purposes. I used RPi pins 32, 36, 33 and 31 for my frets, in order from the lower to the higher frets. The order is important since the software will search for the highest fret in contact with a string at any time. This is to emulate how the highest contacted fret determines the note that is played on a real guitar. I used pins 11, 13, 15, 12, 16 and 18 for the E, A, D, G, B, E strings respectively.

Finally, make sure to wrap any exposed twisted wire ends in a layer of electrical tape so that they are isolated.

Step 7: Setting Up the Raspberry Pi and Getting Ready for Programming

Raspberry Pi Imager v2.0.3

Setup steps

Device

OS

Storage

Customisation

Writing

Done

Choose operating system

Select an operating system to install on your Raspberry Pi



Raspberry Pi OS (32-bit)

A port of Debian Trixie with the Raspberry Pi Desktop (Recommended)

Cached on your computer

Released: 2025-12-04



Raspberry Pi OS (Legacy, 32-bit)

A port of Debian Bookworm with security updates and desktop environment

Online - 1.1 GB download

Released: 2025-11-24



Raspberry Pi OS (other)

Other Raspberry Pi OS based images

APP OPTIONS

BACK

NEXT

Raspberry Pi

It is almost time to program the instrument to generate MIDI, but first we need to make sure we have all the necessary setup and configuration out of the way.

These days, it is so easy to setup a Raspberry Pi. You just need a USB charger, an SD card, USB to SD card adapter, and your Pi in terms of hardware. You can use the official Raspberry Pi Imager to create the bootable drive on the SD card. No more need for Rufus!

Plug the SD card into your computer via the USB to SD card adapter. Select the recommended OS for your Pi in the Raspberry Pi Imager, I installed Trixie (the latest). the RPi OS images are named after Toy Story characters, which is delightful. Follow the steps in the software to image the card.

Once complete, eject the SD and plug it into your Pi. Now is when your peripherals will come in handy. Plug the USB mouse and keyboard into the Pi. Use the HDMI cord to connect the Pi to your monitor. This will allow you to program the Pi. Note that you could also program the Pi from your laptop, but that approach will not be covered in this instructable. Finally, you can connect the Pi to power.

You should see the Pi boot up. You can login using the password you setup in the Imager. The first thing to do in the Pi is to open the terminal and run the following to update your Pi to all the latest software:

```
sudo apt update
```

```
sudo apt full-upgrade
```

Arduino IDE

We will also need to program the Flora using the Arduino IDE. In order to do so, we will need to add support for Adafruit boards like the Flora. You can download the IDE from this official [webpage](#). We will cover how to add support for the Flora next.

Open the preferences menu. We will be adding a URL to the Additional Boards Manager URLs option. Paste **`https://adafruit.github.io/arduino-board-index/package_adafruit_index.json`** into the field for Additional Boards URLs. Click ok to commit the change.

Next, we will open the Board Manager in Tools -> Boards. Under Type -> Contributed, install the **Adafruit AVR Boards**. After installing this, we need to restart the IDE. You may need to install some drivers or enable some permissions. Depending on your operating system, you can find those [here](#) for Windows, [here](#) for Mac or [here](#) for Linux.

Since we are using an LSM303 module in this project, we will also need to install the **Adafruit LSM303 Accel** library using the Library Manager in the IDE. You can find it on the left of the IDE window, click the icon that looks like a collection of books. Then search **Adafruit LSM303 Accel** and click install.

Step 8: Programming the Raspberry Pi

```
MULTIPLIER = 1 / 50

position = 0
bend = 0

# SPI SETUP

# Open SPI bus 0, device (CS) 0
spi = spidev.SpiDev()
spi.open(0, 0)

# Set SPI speed and mode
spi.max_speed_hz = 5000
spi.mode = 0

# Setup as MIDI out device
mido = mido.MidiOut(
    'fluidsynth',
    available MIDI output ports:", midi_ports)

# Find FluidSynth port
found = False
for port in midi_ports:
    if 'fluid' in port.lower():
        found = True
        break
    else:
        continue
except IOError:
    print("Could not open FluidSynth port. Make sure fluidsynth is running.")
    exit()

# If not found, try to find FluidSynth port
if not found:
    print("Could not find FluidSynth port. Make sure fluidsynth is running.")
    exit()

# Array of default MIDI notes
midi_notes = [
    # 40, # open E
    41, # F
    42, # F#
    43, # G
    44, # G#
    # 45, open A
    46, # A#
    47, # B
    48, # C
    49, # C#
    # 50, open D
    51, # D#
    52, # E
    53, # F
    54, # F#
    # 55, open G
    56, # G#
    57, # A
    58, # A#
    59, # B
    # 59, open B
    60, # middle C
    61, # C#
    62, # D
    63, # D#
    # 64, open E
    65, # F
    66, # F#
    67, # G
    68 # G#
```

```
gpio.setmode(gpio.BOARD)

fret_pins = [
    32,
    36,
    35,
    31
]

string_pins = [
    11,
    15,
    15,
    12,
    16,
    18
]

string_inputs = []

for pin in string_pins:
    gpio.setup(pin, gpio.IN, pull_up_down=gpio.PUD_DOWN)
    string_inputs.append(pin)

fret_outputs = []

for pin in fret_pins:
    gpio.setup(pin, gpio.OUT)
    fret_outputs.append(pin)
```

```
-1
= [
    False, False, False],
    False, False, False],
    False, False, False],
    False, False, False],
    False, False, False],
    False, False, False]

found = copy.deepcopy(fret_found)
```

```
errrupt:
```

```
or advance to final step
) % 5
```

```
h_pos != 4:
    Scan to next fret
    for pos, fret_pin in enumerate(fret_pins):
        if pos == high_pos:
            gpio.output(fret_pin, gpio.HIGH)
        else:
            gpio.output(fret_pin, gpio.LOW)

    Read the input from the mini guitar neck
    for i in range(6):
        inp = string_inputs[i]
        if gpio.input(inp):
            fret_found[i][high_pos] = True
        else:
            fret_found[i][high_pos] = False
```

```
if high_pos == 4:

    # We have completed a scan
    # Here we check if multiple fret positions are contacted,
    # or if multiple notes are played i.e. in a chord

    # Read from SPI... It takes a few tries
    last_position = position
    last_bend = bend
    bend = None
    position = None

    while (bend is None) or (position is None):
        read = 255
        tries = 0
        while read > 100 and tries < 10:
            read = spi.xfer2([i])[0]
            time.sleep(0.01)
            tries += 1

        if tries == 10:
            position = last_position
            bend = 0

        # Figure out which response is position and which is bend
        elif 0 <= read <= 5:
            position = read
        else:
            bend = read
```

```

# Check for multiple fret contacts
note_indices = []

for i in range(6):
    j = 0

    # Find right-most contact for a string
    max_high = -1
    while j < 4:
        if fret_found[i][j] and j > max_high:
            max_high = j
            j += 1

    # Remove irrelevant contacts (probably the one immediately to the left)
    for k in range(4):
        if k == max_high:
            fret_found[i][k] = True
        else:
            fret_found[i][k] = False

    if last_fret_found[i][max_high] == False and fret_found[i][max_high] == True:
        # Calculate the played note index (without position, bend)
        note_indices.append(i * 4 + max_high)

```

```

position and any bend
= int(midi_notes[note_index] + 4 * po
e = int(midi_notes[note_index] + 4 * p
)

```

The software for this project can all be found on GitHub in the open source [repository](#), but the whole point of an instructable is to do it yourself, so let's go through the logic together!

Raspberry Pi Dependencies

Prior to running the code on the Raspberry Pi, you will need to install some dependencies.

```

sudo apt-get install pulseaudio fluidsynth python3-spidev
pip install mido

```

Then we need to enable SPI. Run the following and then navigate to Interface Options, where you can choose to enable SPI.

```

sudo raspi-config

```

After enabling SPI, you will need to reboot the Pi.

```

reboot

```

Once we are booted and logged back into the OS, we can open Geany and begin to write the code that will generate our MIDI!

Raspberry Pi Code Walkthrough

The first thing we do in any Python script that relies on third part code is import the relevant packages. We will need **spidev** to talk to the Flora over SPI, **time** to debounce our instrument's strings in a similar way to how we would debounce a button, **copy** so we can compare our finger positions at a given time to what they were before, **RPi.GPIO** so we can do I/O and finally **mido** to generate the MIDI notes.

Another common step at the beginning of a script is to declare and/or initialize any global variables. In our case, we will want to initialize variables for the position our 4 frets map to on an imaginary full-length guitar fretboard. We will rely on the Flora to tell us when we move up and down the imaginary fretboard by quick motions of our playing hand to the left and right which will shift the notes up and down by 4 fret positions each time. This is similar to the octave up and octave down buttons you might find on an off-the-shelf MIDI controller.

We will also want to know if we are "bending" a note at any given time by moving the instrument up and down. The Flora will tell us this too, and we can translate it to increased pitch.

Finally, we want to initialize a constant for the multiplier that will translate the extent of a bend to the corresponding increase in pitch.

Next, we'll do the SPI setup. We will only have one device on the SPI bus, so the chip select doesn't matter so much. We will open device zero. We also have to set the mode and the speed. The speed is the speed at which data is transferred. This determines how fast we pulse the SCK clock. The mode tells us whether we read and write data on a high-to-low edge or vice versa, etc. It is just important that the speed and mode match what we configure on the Flora later.

Now we can setup the MIDI. How this will work is the MIDI will be sent by **mido** to an open port internal to the RPi that will allow us to stream the notes to **fluidsynth**. **Fluidsynth** will then convert the notes to audio that can be played out the RPi's audio jack in a timbre configured by a soundfont file installed on the RPi. We have to check if **fluidsynth** is running and if we can connect to it.

To finish setting up the MIDI, we need to initialize a data structure that will allow us to map the positions played on the instrument to the corresponding notes on a guitar.

We will need to setup **GPIO** too. To use the wiring we describe previously, we will use the following pins. We read from the string pins and write to the fret pins. This will allow us to detect when a string and a fret are in contact: if we can read from a string what we've written to a fret, they are connected. We configure an internal pull-down resistor for each string pin so that when a high signal is not present on the pin, we will read a low one.

There are just a few more variables to setup before we get into the main loop. We need a variable for keeping track of the fret position which we are currently writing high, as we will write each fret high in a scanning pattern. We need a data structure, a 4 x 6 array, to keep track of what finger positions are "pressed" at any given time. We finally need a copy to help us compare the current finger positions to the last fret positions we saw.

Now we can get into the main loop. It should be wrapped in a **try / catch** so that we can gracefully exit if any exceptions occur.

The main loop has three parts. The first part is the simplest and happens every time. In it, we sleep for a short time to debounce, and we increment the high position.

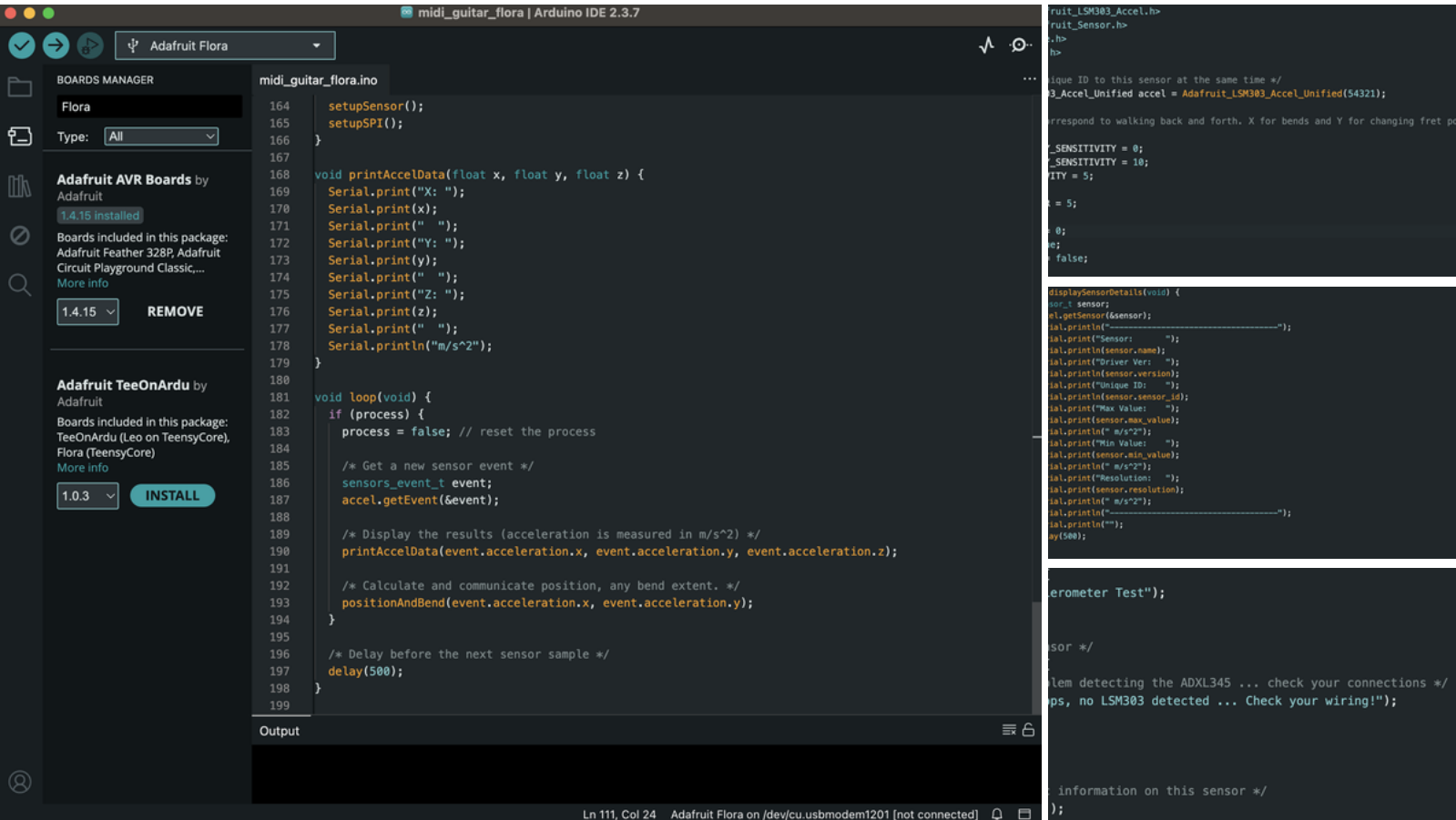
The other two parts of the loop are branches controlled by the **high_pos**. The program consists of 5 steps which are repeated infinitely. The first four steps each write one fret high and all the others low, then read from the strings to determine which strings (if any) are contacting the high fret.

The final step occurs after each scan of the four frets is complete. We first read from the SPI. Since the RPi is the master, it needs to initiate every communication with the Flora. So, we write send a meaningless byte to the Flora and expect its response. It sometimes takes a few tries. This might be a quirk of my particular LSM303 and Flora, but with the retry loop it typically works. We classify the response we get as either an update to the **position**, a **bend**, or an error.

Now we see why we waited for a full scan of the fretboard before doing this final step. It allows us to see if multiple notes are being played at once. We need to find the right-most (left-most if you play with your right hand) contact that each string was found to have with each fret since this is the contact that is relevant to which note should be played. We index into the notes array to get the MIDI note itself.

Once we're done with the **note_indices**, we can play the MIDI notes over the **fluidsynth** port. We finally update the last positions we've seen.

Step 9: Programming the Flora



```
void setupSensor(void) {
  sensorTest();

  accel.setRange(LSM303_RANGE_4G);
  Serial.print("Range set to: ");
  ls303_accel_range_t new_range = accel.getRange();
  switch (new_range) {
    case LSM303_RANGE_2G:
      Serial.println("-- 2G");
      break;
    case LSM303_RANGE_4G:
      Serial.println("-- 4G");
      break;
    case LSM303_RANGE_8G:
      Serial.println("-- 8G");
      break;
    case LSM303_RANGE_16G:
      Serial.println("-- 16G");
      break;
  }

  accel.setMode(LSM303_MODE_NORMAL);
  Serial.print("Mode set to: ");
  ls303_accel_mode_t new_mode = accel.getMode();
  switch (new_mode) {
    case LSM303_MODE_NORMAL:
      Serial.println("Normal");
      break;
    case LSM303_MODE_LOW_POWER:
      Serial.println("Low Power");
      break;
    case LSM303_MODE_HIGH_RESOLUTION:
      Serial.println("High Resolution");
      break;
  }
}
```

```
routine */
) {

  // read byte from SPI Data Register
  ;

  id)

  E); // turn on SPI in slave mode
  OUTPUT);
  INPUT);
  INPUT);
  rrupt(); // turn on interrupt
  PUT);
}
```

```
void positionAndBend(float x, float y) {
  if (pos) {
    /* Negative y dimension increases fret position. */
    if (y > POSITIVE_Y_SENSITIVITY) {
      position = position - 1;
    } else if (y < -NEGATIVE_Y_SENSITIVITY) {
      position = position + 1;
    }

    /* Roughly five 4-fret positions on a guitar neck. */
    if (position > 5) {
      position = 5;
    } else if (position < 0) {
      position = 0;
    }
    SPQR = int(position);
  } else {
    int extent = 0;
    if ((x > X_SENSITIVITY) || (x < -X_SENSITIVITY)) {
      extent = x;
    }

    /* Scale the bend extent. */
    extent = extent * MULTIPLIER;

    /* No negative bends, as they don't exist on real guitars. */
    if (extent > 100) {
      extent = 100;
    } else if (extent < 0) {
      extent = 0;
    }
    SPQR = int(extent);
  }
}
```

```
Leonardo, etc until serial console opens
```

```
x, float y, float z) {

  // reset the process

  sensor event */
  event;
  event);

  results (acceleration is measured in m/s^2) */
  event.acceleration.x, event.acceleration.y, event.acceleration.z);

  d communicate position, any bend extent. */
  event.acceleration.x, event.acceleration.y);

  he next sensor sample */
}
```

Flora Code Walkthrough

Now we will take a look at the Processing code running on the Flora. The code is mainly broken up into a setup function, main loop and helper functions.

The setup involves setting up any Serial communication with your computer that we may be using for debugging, setting up the communication with the accelerometer and setting up the SPI.

The SPI setup involves calling **SPI.begin** or **SPI.beginTransaction**. This is where we can make sure that the SPI on the Flora is configured with the same speed and mode as the SPI config on the RPi side. We also need to write certain bits of the **SPCR** register to set the SPI to be inslave mode. We also need to set the following pin modes: Master In Slave Out (**MISO**) is an output, Master Out Slave In (**MOSI**) is an input and clock (**SCK**) is an input. Common sense.

Crucially (and this doesn't seem well documented anywhere), we also need to configure the chip select pin (**SS**) as an input or the Flora won't operate in SPI slave mode. We have to do this even though the **SS** pin is inaccessible on the Flora and is connected to one of the onboard LEDs.

We finally need to provide an interrupt subroutine named **ISR** which will be called whenever there is incoming data over SPI. In it, we will read a byte from SPI data register (for debugging purposes), toggle a flag that will determine what sort of data (bend or position) we will send as a response this time, and set another flag to tell the main loop we have had a request for data from the RPi.

In the sensor setup, we start by making sure we can connect to the LSM303 by running **accel.begin**. We can print some details about the sensor over Serial when we connect. We then set the range and the mode of the accelerometer. For simple gesture detection, we can use the Normal mode. We don't need Low Power or High Resolution.

In the main loop, we check in each loop iteration whether the **process** flag has been set by the **ISR**. If it has, then we read from the accelerometer using **accel.getEvent**. Then, we print the associated sensor data (x, y, z) for debugging purposes. Finally, we call a helper function to calculate the **position** and **bend** extent.

The helper function has two main branches, one for calculating and communicating the **position** and one for the **bend**. The position simply gets incremented whenever we detect a negative acceleration above a certain threshold, or decremented whenever we detect a positive acceleration above a threshold. We then write the **position** to the SPI data register.

We register a bend whenever an x-dimension motion is detected above a certain threshold. The bend is always positive and the speed of the bend determines the value that is sent to the Pi.

At the end, we reset the **process** flag.

Step 10: Conclusion

Now it's time to play your hand-made guitar MIDI controller!

Running the Code

To run the code, the RPi and Flora need to be plugged in. You could also power them using a battery, but that would be an improvement beyond the scope of this instructable. To run the code on the Pi, we need to first start **fluidsynth**. Then we can run the **main.py** script.

```
fluidsynth -a alsa -m alsa_seq -g 1.0 <path-to>/60s_Rock_Guitar.SF2 &  
python3 main.py
```

The Flora will start to run the code you've flashed to it immediately whenever you power it on.

Playing

You can hook the aux cable from your Raspberry Pi into your speaker or - even better - into your pedal board to add effects. Since the project uses MIDI, there is also no limit to the sounds the controller can make! Just download a different soundfont and use *i* in place of the one provided in the GitHub to make your instrument sound like whatever you desire.

Below is a short video of me playing a riff on mine!

Discussion

My prototype of the instrument works and it is super fun to work on and play, but it is far from perfect. It is finicky to play as you have to manage the weight of the head, holding the instrument up, making gestures, and playing all with the same hand. Furthermore, it is hard to reach every position on the fingerboard since your hand is strapped to the neck. On a usual guitar, you can move your hand freely around the neck. This project helped me realize how much I actually make use of this fact to play a normal guitar!

It would excite me to no end if someone were to improve upon this design. Thanks so much for reading!